List of Experiments

1. SolveTic-Tac-ToeusingDepthFirstSearch(DFS).
2. 8-PuzzleUniqueStates&InversionParityAnalysis.
3. PredicateLogic&KnowledgeRulesRepresentation.
4. ConceptLearning(Find-SandCandidateElimination).
5. DecisionTreeConstructionusingID3Algorithm.
6. ID3PerformanceonDatasetsofVaryingComplexity.
7. DemonstrationofCoreMachineLearningParadigms.
8. Hypothesis Space SearchinConceptLearning.
9. CompleteMachineLearningRegressionPipeline.
10. Binary&MulticlassClassificationusingCNNonMNIST.

Experiment1:Tic-Tac-ToeProblemUsingDepthFirstSearch(DFS)

Aim: SolveTic-Tac-ToeusingDFS techniqueinPythontodeterminetheoptimaloutcomeassumingperfectplaybyboth players.

PythonCode

```
defis_winner(board,player):
    win_states=[(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    returnany(all(board[i]==playerforiinstate)forstateinwin_states)

defdfs(board,player):
    if is_winner(board,'X'):return1 elif
    is_winner(board,'O'):return -1 elif
    ''notin board:return 0

    scores=[]
    for i in range(len(board)):
        if board[i]== '':
            board[i]= player
            score=dfs(board,'O'ifplayer=='X'else'X')
            scores.append(score)
            board[i]=''
    returnmax(scores)ifplayer=='X'elsemin(scores)

#PlayerXforcewincase
board = ['X','X','','O','O','','','','']
print("DFSResult:",dfs(board,'X'))
```

```
DFSResult:1 →PlayerXcanforceaWIN.
```

Experiment2:IdentifyandDisplayUnique8-PuzzleStates

Aim: Extractunique8-Puzzleconfigurationsandclassifystatesintoreachabilitypartitionsbasedoninversionparity.

PythonCode

```
defdisjoint_sets(states):
    return{tuple(s)forsinstates}

defcount_inversions(state):
    flat=[tilefortileinstateiftile!=0]
    returnsum(1foriinrange(len(flat))forjinrange(i+1,len(flat))ifflat[i]>flat[j])

defclassify_states(states):
    even,odd=[],[]
    forsinstates:
        (even if count_inversions(s) %2 == 0 else odd).append(s)
    returneven, odd

states=[[1,2,3,4,5,6,7,8,0],[1,2,3,5,4,6,7,8,0]]
even_set,odd_set = classify_states(states)
print("Even Inversion States:",even_set)
print("Odd Inversion States:",odd_set)
```

```
EvenInversionStates:[[1,2,3,4,5,6,7,8,0]]OddInversionStates:[[1,2,3,5,4,6,7,8,
0]]
```

Experiment3:PredicateLogic&KnowledgeRulesRepresentation

Aim: Represents cenarios using predicate logic, store facts/rules, and evaluate queries using Python.

PythonCode

```
classKnowledgeBase:
    definit(self):
        self.facts={}
        self.rules={}

    defadd_fact(self,predicate,value):
        self.facts.setdefault(predicate,set()).add(value)

    defadd_rule(self,predicate,rule):
        self.rules.setdefault(predicate,[]).append(rule)

    defevaluate(self,predicate,value):
        ifvalueinself.facts.get(predicate,set()):returnTrue
        forruleinself.rules.get(predicate,[]):
            ifrule['condition'](self,value):returnrule['conclusion'](self,value)
        returnFalse

kb=KnowledgeBase()
kb.add_fact('mammals','dog')
kb.add_fact('birds','sparrow')
kb.add_rule('birds',{'condition':lambdak,a:aink.facts.get('birds',set()),
                     'conclusion':lambdak,a:True})

print("sparrowisbirdandcanfly:",kb.evaluate('birds','sparrow'))
```

```
sparrowisbirdandcanfly:True
```

Experiment4:ConceptLearningUsingFind-SandCandidateElimination

Aim: Apply Find-S and Candidate Elimination algorithms to a concept learning task and compare outputs.

PythonCode

```
#Find-SAlgorithm
deffind_s(instances,positive_indices):
    S=['Null']*6
    for idx in positive_indices:
        x=instances[idx]
        foriinrange(len(S)):
            ifS[i]=='Null':S[i]=x[i]
            elifS[i]!=x[i]:S[i]='?'
    returnS

instances=[
    ['sunny','warm','normal','strong','warm','same'],
    ['sunny','warm','high','strong','warm','same'],
    ['rainy','cold','high','strong','warm','change'],
    ['sunny','warm','high','strong','cool','change']
]

S=find_s(instances,[0,1,3])
print("LearnedSpecificHypothesisS:",S)
```

```
LearnedSpecificHypothesisS:['sunny','warm','?','strong','?','?']
```

Experiment5:DecisionTreeUsingID3Algorithm

Aim: ConstructadecisiontreeusingID3algorithmonasimpleweatherclassificationdataset.

PythonCode

```
importpandasaspd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier

data= {
    'Outlook':['Sunny','Sunny','Overcast','Rain','Rain','Overcast'],
    'Temperature':['Hot','Hot','Hot','Mild','Cool','Mild'],
    'Humidity':['High','High','High','High','Normal','Normal'],
    'Windy':[False,True,False,False,False,True],
    'Play':['No','No','Yes','Yes','Yes','Yes']
}

df = pd.DataFrame(data)
le = LabelEncoder()
forcolindf.columns:df[col]=le.fit_transform(df[col])

X= df.drop('Play',axis=1) y =
df['Play']

clf=DecisionTreeClassifier(criterion='entropy',random_state=42)
clf.fit(X,y)
print("ModelScore:",clf.score(X,y))
```

```
ModelScore:1.0
```

Experiment6:PerformanceAnalysisofID3DecisionTree

Aim: AssessID3performanceondatasetswithvaryingcomplexity,examiningunderfittingandoverfittingacrossdepths.

PythonCode

```
fromsklearn.datasetsimportmake_classification,make_moons,make_circles
fromsklearn.treeimportDecisionTreeClassifier from
sklearn.model_selection import train_test_split from
sklearn.metrics import accuracy_score

datasets={
    "Linear":make_classification(n_samples=100,n_features=2,n_redundant=0,random_state=42),
    "Moons":make_moons(n_samples=100,noise=0.1,random_state=42),
    "Circles":make_circles(n_samples=100,noise=0.1,factor=0.5,random_state=42)
}

forname,(X,y)indatasets.items():
    X_tr,X_te,y_tr,y_te      =      train_test_split(X,y,test_size=0.3,random_state=42)
    clf=DecisionTreeClassifier(max_depth=4,criterion='entropy',random_state=42)
    clf.fit(X_tr,y_tr)
    print(f"{name}TestAccuracy(Depth4):",accuracy_score(y_te,clf.predict(X_te)))
```

```
LinearTestAccuracy(Depth4):0.967MoonsTestAccuracy(Depth4):0.933CirclesTestAccuracy
(Depth4):1.000
```

Experiment7:DemonstrationofMachineLearningApproaches

Aim: ImplementSupervised,Unsupervised,Semi-Supervised,andReinforcementLearningapproachesonasampletask.

PythonCode

```
importnumpyasnp
fromsklearnimportdatasets
from sklearn.tree import DecisionTreeClassifier
fromsklearn.clusterimportKMeans
fromsklearn.semi_supervisedimportLabelSpreading

#1.Supervised
iris=datasets.load_iris()
clf=DecisionTreeClassifier().fit(iris.data,iris.target)

#2.Unsupervised
kmeans=KMeans(n_clusters=3,random_state=42).fit(iris.data)

#3.Semi-Supervised
y_semi=np.copy(iris.target)
y_semi[::2]=-1#Mask50%
ls=LabelSpreading().fit(iris.data,y_semi)

#4. Reinforcement Learning simulation
rewards = {"Start":0,"Mid":1,"Goal":10} state,
total_reward="Start", 0
forsin["Mid","Goal"]:
    state=s
    total_reward+=rewards[state]

print("SupervisedScore:",clf.score(iris.data,iris.target))
print("UnsupervisedClusters(First5):",kmeans.labels_[:5])    print("RL
Simulation Final Reward:",total_reward)
```

```
SupervisedScore:1.0UnsupervisedClusters(First5):[11111]RLSimulationFinalReward:11
```

Experiment8:HypothesisSpaceSearchinConceptLearning

Aim: AnalyzehowFind-SandCandidateEliminationsearchthroughthehypothesisspaceontheEnjoySportsdataset.

PythonCode

```
importnumpyasnp

data=np.array([
    ['Sunny','Warm','Normal','Strong','Warm','Same','Yes'],
    ['Sunny','Warm','High','Strong','Warm','Same','Yes'],
    ['Rainy','Cold','High','Strong','Warm','Change','No'],
    ['Sunny','Warm','High','Strong','Cool','Change','Yes']
],dtype=object)

# Generalize specific hypothesis S
S = list(data[0][:-1])
forrowindata:
    ifrow[-1]=='Yes':
        forjinrange(len(S)):
            ifS[j]!=row[j]:S[j]='?'

print("FinalSpecificHypothesisS:",S)
```

```
FinalSpecificHypothesisS:['Sunny','Warm','?','Strong','?','?']
```

Experiment9:CompleteMachineLearningRegressionPipeline

Aim: Implementanend-to-endregressionpipelineontheCaliforniaHousingdatasetusingLinearRegressionandRandom Forest.

PythonCode

```
importnumpyasnp
fromsklearn.datasetsimportfetch_california_housing
fromsklearn.model_selectionimporttrain_test_split,GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

housing=fetch_california_housing(as_frame=True)
X_tr,X_te,y_tr,y_te=train_test_split(housing.data,housing.target,test_size=0.2,
random_state=42)

#LinearRegressionBaseline
lin_reg=LinearRegression().fit(X_tr,y_tr)
lin_rmse=np.sqrt(mean_squared_error(y_te,lin_reg.predict(X_te)))

#RandomForestwithHyperparameterTuning rf =
RandomForestRegressor(random_state=42)
grid=GridSearchCV(rf,{'n_estimators':[10,30],'max_features':[4,6]},cv=3)
grid.fit(X_tr,y_tr)
rf_rmse=np.sqrt(mean_squared_error(y_te,grid.best_estimator_.predict(X_te)))

print(f"LinearRegressionRMSE:{lin_rmse:.2f}")
print(f"RandomForestRMSE:{rf_rmse:.2f}")
```

```
LinearRegressionRMSE:0.75RandomForestRMSE:0.51
```

Experiment10:Binary&MulticlassDigitClassification(CNNonMNIST)

Aim: Performbinary (3vs8)andmulticlassclassificationonMNIST datasetusingCNN,evaluatingperformancemetrics and error analysis.

PythonCode

```
importnumpyasnp
fromkeras.datasetsimportmnist from
keras.models import Sequential
fromkeras.layersimportConv2D,MaxPooling2D,Flatten,Dense
fromkeras.utilsimportto_categorical from
sklearn.metrics import accuracy_score

(X_tr,y_tr),(X_te,y_te)=mnist.load_data()
X_tr=X_tr.reshape((-1,28,28,1)).astype('float32')/255.0
X_te=X_te.reshape((-1,28,28,1)).astype('float32')/255.0

#1.BinaryClassification (3 vs 8)
bin_tr = np.isin(y_tr,[3,8]) bin_te =
np.isin(y_te,[3,8])

model_bin=Sequential([
    Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(32,activation='relu'),
    Dense(1,activation='sigmoid')
])
model_bin.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
model_bin.fit(X_tr[bin_tr],(y_tr[bin_tr]==8).astype(int),epochs=2,batch_size=128,verbose=0)
pred_bin=(model_bin.predict(X_te[bin_te])>0.5).astype(int)
print("Binary(3vs8)Accuracy:",accuracy_score((y_te[bin_te]==8).astype(int),pred_bin))

#2.MulticlassClassification(0-9)
model_multi=Sequential([
    Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(64,activation='relu'),
    Dense(10,activation='softmax')
])
model_multi.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model_multi.fit(X_tr, to_categorical(y_tr, 10), epochs=2, batch_size=128, verbose=0) pred_multi
= np.argmax(model_multi.predict(X_te),axis=1)
print("Multiclass(10Digits)Accuracy:",accuracy_score(y_te,pred_multi))
```

```
Binary(3vs8)Accuracy:0.9950Multiclass(10Digits)Accuracy:0.9825
```

LabExecutionSummary:

All10experimentshavebeenexecutedandverifiedinPython3.x/GoogleColabwithoutputvalidations.